



LOST BOYS CYBER

THREAT INTELLIGENCE • MALWARE ANALYSIS • DETECTION ENGINEERING



AsyncAPI npm Supply Chain Compromise Shows Import-Time Payload Risk

Threat Report

Classification	TLP:CLEAR
Published	2026-07-28
Version	1.0
Author	Lost Boys Cyber
Report Type	Threat Report

TLP:CLEAR | Lost Boys Cyber | AsyncAPI npm Supply Chain Compromise Shows Import-Time Payload Risk - Threat Report

Published: 2026-07-28 | TLP:CLEAR | Version: 1.0 | Author: Lost Boys Cyber

AI Generation: This report was generated with AI assistance and is provided for informational purposes only. All findings, IOCs, detection rules, hunting queries, attribution assessments, and recommendations must be independently reviewed and validated by a qualified security professional before operational use.

Table of Contents

1. Executive Summary
2. Intelligence Requirements
3. Source Review & Confidence
4. Threat Activity Overview
5. Affected Sectors and Exposure
6. Tactics, Techniques, and Procedures
7. Infrastructure and Indicators
8. Detection Engineering
 - 8.1. Detection Summary
 - 8.2. Hunt Query
9. Threat Hunting
 - 9.1. Hunt Hypothesis
 - 9.2. Hunt Query
10. Risk Assessment
11. Recommended Actions
12. References

AsyncAPI npm Supply Chain Compromise Shows Import-Time Payload Risk

1. Executive Summary

Microsoft Threat Intelligence reported a coordinated compromise of the @asynapi npm organization identified on 2026-07-14 and publicly analyzed on 2026-07-15. The attack reportedly abused CI/CD trust and delivered malicious package payloads that execute at import time rather than relying only on traditional npm install scripts.

This matters to defenders because package-install controls that focus on lifecycle scripts may not catch payloads triggered when a developer, build system, or application imports the compromised library. The incident reinforces that software supply-chain defense must cover repository workflows, publishing tokens, package provenance, dependency pinning, import/runtime behavior, and build telemetry.

2. Intelligence Requirements

Question	Current Answer	Confidence
What was targeted?	The @asynapi npm organization and related packages used around AsyncAPI specification/code generation workflows.	High
What made the technique notable?	Public reporting says payload delivery occurred at import time, weakening install-script-only safeguards.	High
Which environments should prioritize review?	Developer workstations, CI runners, package mirrors, and applications that installed or imported affected AsyncAPI packages around 2026-07-14.	Medium

3. Source Review & Confidence

Source	Contribution	Confidence
Microsoft Threat Intelligence	Primary technical analysis and timeline.	High
The Hacker News	Secondary reporting on affected packages and GitHub Actions/pull_request_target context.	Medium
Wiz M-Red-Team	Additional CI/CD and token-abuse context from public reporting.	Medium

4. Threat Activity Overview

The reported intrusion centered on a trusted open-source package namespace. Microsoft stated that the compromise was coordinated and involved malicious code pushed through the npm ecosystem. Secondary reporting links the origin to a pwn request against the asynapi/generator repository caused by a misconfigured GitHub Actions workflow using pull_request_target, followed by abuse of a compromised asynapi-bot personal access token.

5. Affected Sectors and Exposure

Exposure Area	Operational Relevance
Software engineering teams	Direct exposure if affected packages were installed, imported, or

	built in CI.
CI/CD infrastructure	Build logs, runner filesystems, tokens, and package caches may hold evidence or secondary secrets.
Downstream application owners	Runtime import behavior means application execution paths may matter, not only package installation time.

6. Tactics, Techniques, and Procedures

Tactic / Phase	Observed Technique	Evidence
Initial Access	Trusted relationship / supply-chain compromise	Compromised package namespace and GitHub Actions/publishing workflow.
Execution	Import-time code execution	Microsoft/public reporting emphasizes import-time payload delivery.
Credential Access	Token abuse risk	Reporting references a compromised service-account PAT used to push malicious commits.

7. Infrastructure and Indicators

Indicator / Infrastructure	Type	Operational Use
@asynapi npm organization packages	Package namespace	Dependency inventory and package-lock review.
asynapi/generator workflow context	Repository/workflow	CI/CD audit pivot for pull_request_target abuse.
asynapi-bot PAT references	Credential/service account	Token rotation and audit pivot.

8. Detection Engineering

8.1. Detection Summary

Signal	Telemetry Source	Analytic Goal
Package installation or import of affected @asynapi packages	CI logs, package manager logs, EDR file telemetry	Identify exposed hosts/runners.
GitHub Actions pull_request_target use with write-capable tokens	GitHub audit/workflow configuration	Find comparable workflow misconfigurations.
Network/process execution from build runners after package import	EDR, proxy, DNS	Surface payload execution and callback attempts.

8.2. Hunt Query

```
DeviceFileEvents
| where Timestamp between (datetime(2026-07-14)..now())
| where FolderPath has_any ("node_modules/@asynapi", "package-lock.json", "pnpm-lock.yaml",
"yarn.lock")
| summarize Hits=count(), FirstSeen=min(Timestamp), LastSeen=max(Timestamp) by DeviceName, FolderPath,
InitiatingProcessFileName
| order by LastSeen desc
```

9. Threat Hunting

9.1. Hunt Hypothesis

Compromised package execution on developer or CI systems should produce a cluster of dependency-file changes followed by unusual child processes, network callbacks, or credential access from build tools.

9.2. Hunt Query

```
DeviceProcessEvents
| where Timestamp between (datetime(2026-07-14)..now())
| where InitiatingProcessCommandLine has_any ("npm", "pnpm", "yarn", "node", "tsx", "vite", "webpack")
| where FileName in~ ("sh", "bash", "curl", "wget", "python", "node")
| project Timestamp, DeviceName, AccountName, FileName, ProcessCommandLine,
InitiatingProcessCommandLine
| order by Timestamp desc
```

10. Risk Assessment

Likelihood is highest for engineering teams that installed or imported @asynccapi packages during the compromise window. Impact depends on what secrets were available to affected workstations/runners and whether package import occurred in privileged build contexts.

11. Recommended Actions

Priority	Owner	Action
Critical	Engineering platform	Identify affected package versions and rebuild from known-good dependencies.
Critical	IAM / DevOps	Rotate tokens available to impacted CI runners and developer environments.
High	AppSec	Audit pull_request_target workflows and enforce least-privilege GitHub tokens.

12. References

- Microsoft Threat Intelligence report on AsyncAPI npm supply-chain compromise.
- The Hacker News coverage of compromised AsyncAPI npm packages.
- Wiz M-Red-Team reporting on GitHub Actions and token-abuse context.